

메소드 : 기능.zip, 명령.zip, 함수.

컴퓨터가 특정 기능을 수행하도록 명령하는 코드 블록에 이름을 붙인 것.

* 표현식

A B C D E
접근제한자 예약어 반환형 메소드명 (매개변수) {

// 기능 정의.

// 반환(반환형 != void) {

return 데이터; 작성필수

}

A. public (+) : from anywhere

protected (#) : 동일 패키지 + 상속관계 클래스

default (~) : 동일 패키지. 접근제한자 자리를 비워두면 default.

private (-) : 동일 클래스.

B. static : 객체 생성 없이 메소드 사용 가능. (static 영역에 바로 올림)

final : 기능의 고정. 네브 실행코드 수정 불가 (상수처럼!)

abstract : 미완성 메소드.

synchronized : 동기화. 여러 작업이 몰릴 때 충돌 방지.

예약어는 생략 가능.

C. void : 반환형 없음. (return ; 으로 메소드 강제종료 가능.)

자료형 : 해당 자료형의 값을 반환.

D. 명명관계는 변수명 짓기 관계와 거의 같다. ⇒ camelCase.

E. 매개변수 X ⇒ 바워둬기.

자료형 변수명은 소괄호 안에 작성.

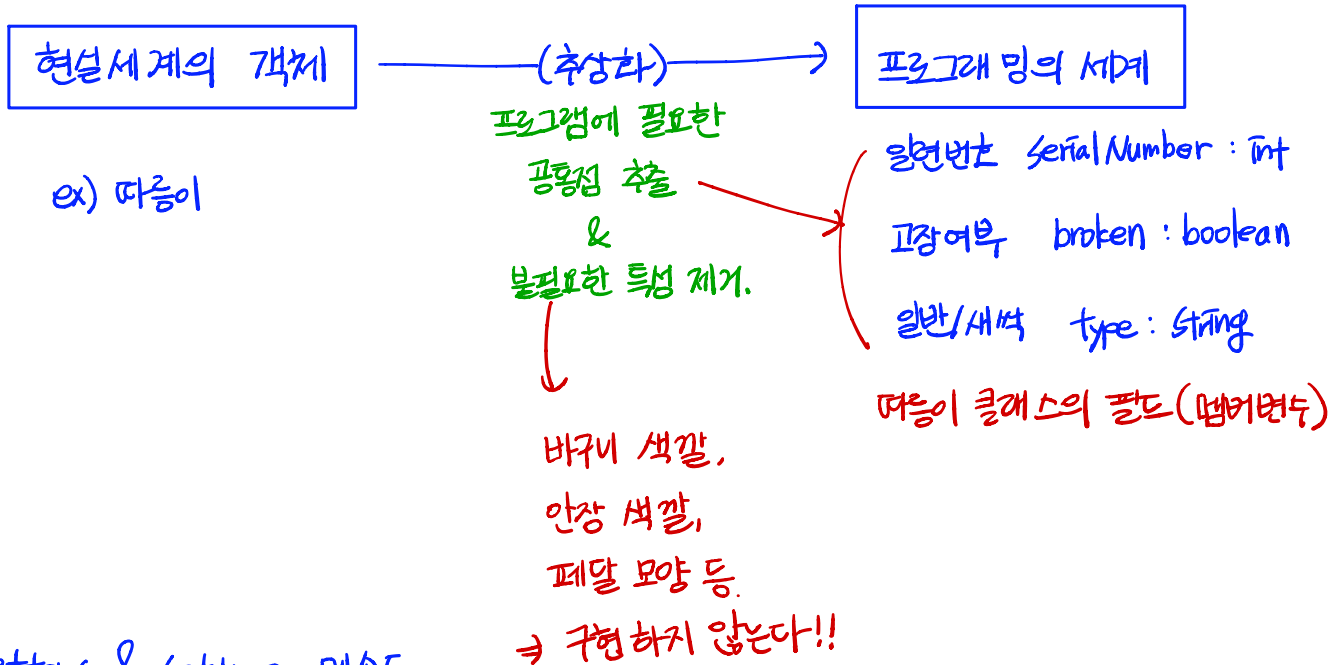
매개변수 정보(개수, 자료형, 순서)가 다르면 메소드 오버로딩 조건 충족.

한 클래스 안에 동일한
이름의 메소드를 여러 개
정의하는 것.

* 메소드 호출시 코드 진행 흐름

위→아래, 왼쪽→오른쪽 (대입연산자 제외.) 방향으로 진행하다가, 메소드 호출문을 만나면 제어권이 일시적으로 메소드에 이동. 메소드 실행 완료 후 호출문 다음으로 가서 마저 진행.

OOP (Object-Oriented Programming: 객체 지향 프로그래밍)



getters & setters 메소드

클래스 내의 멤버변수는 주로 private로 선언하여 외부에서 마음대로 열람·수정할 수 없게 한다. 객체 간 상호작용을 위해서는 정보에 접근이 가능해야 하므로 메소드를 통해 간접적인 접근을 가능하게 한다.

ex) 멤버변수 serialNumber

getSerialNumber : 호출 시점에 serialNumber 값을 가져온다.

setSerialNumber : 호출 시점에 serialNumber에 매개변수로 전달받은 값을 저장(대입)한다.

→ 이때 멤버변수와 매개변수명을 똑같이 짓는 경우가 많은데, 구분을 위해 this 키워드를 사용한다.

this.serialNumber = serialNumber ;

↓
이 클래스의 멤버변수

↓
이 메소드의 매개변수.

Person 클래스

```
private String name;
private int age;
private char gender;
```

멤버변수(=필드)

```
public Person ();
public Person (String name);
```

기본생성자. (매개변수 X)
생성자 (매개변수는 name: String)

생성자.
클래스명과 대소문자까지
동일!! 반환형 존재 X

```
public String getName() {
    return name;
}
public void setName(String name) {
    this.name = name;
}
```

메소드

this.name = name;
이 클래스의 매개변수인 name
name

→ 생성자도 메소드.

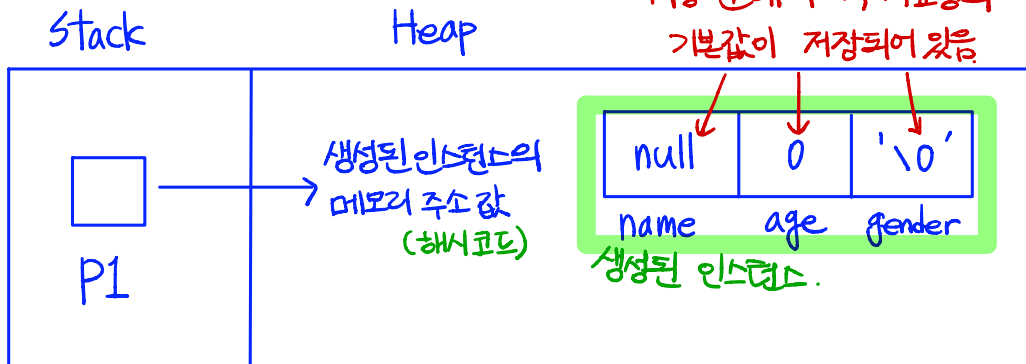
∴ 매개변수 정보가
다르면 생성자
오버로딩이 가능함!

예시) Person p1 = new Person ();

클래스 참조변수

기본생성자

기본생성자로 생성후 데이터를 따로
저장 안해서 각 자료형의
기본값이 저장되어있음



참조변수 p1이 인스턴스의 주소값을 가리키도록 하는 메소드 = 생성자.